

Instance Search Implementation Report

1. Introduction

This report presents the implementation of two object matching methods for instance search, as required by the CS5187 Vision and Image Assignment 1. The task involves retrieving the top 10 most similar images containing the same instance(s) from a collection of 28,493 images for 50 query images. The two methods employed are **VGG16-based Deep Feature Matching** (a deep neural network method) and **Histogram-based Color Feature Matching** (a traditional computer vision method). This report details the methodologies, implementation steps, experimental results, and performance analysis of both methods.

2. Methodology

2.1 VGG16-based Deep Feature Matching

2.1.1 Principle

The VGG16 model, a pre-trained deep convolutional neural network (DCNN) trained on the ImageNet dataset, is used for feature extraction. It learns hierarchical, discriminative features that capture high-level semantic information of images—critical for instance recognition. Cosine similarity measures the similarity between query and collection image features, defined for a query $Q = [q_1, q_2, \dots, q_n]$ and an image $F = [f_1, f_2, \dots, f_n]$ as:

$$\text{Similarity}(Q, F) = \frac{\sum_{i=1}^n q_i f_i}{\sqrt{\sum_{i=1}^n q_i^2}}$$

A higher cosine similarity indicates greater instance similarity.

2.1.2 Implementation Steps

1. Data Preprocessing

- **Cropping:** Using bounding box data from JSON annotations (./Images_json/), target instances in original images (./Images/) and queries (./Queries/) are cropped. Results are saved to ./cropped_images_yolo/ (collection) and ./cropped_queries/ (queries). Images without bounding boxes are saved as full images.
- **Resizing:** Cropped images are resized to 224×224 (VGG16 input size) via bicubic interpolation (cv2.INTER_CUBIC), stored in ./cropped_resized_images_yolo/ and ./cropped_resized_trans_queries/.
- **Augmentation:** For model fine-tuning, query images undergo rotation (0-180°), flipping, shifting (±10%), shearing (±10%), and zooming (90%-150%) to enhance generalization.

1. Model Fine-Tuning

- Load pre-trained VGG16 (without top layers, include_top=False).
- Add custom layers: Flatten (convert 4D features to 2D), 1024-neuron Dense layer (ReLU), Dropout (50% rate, anti-overfitting), and softmax output layer (matches dataset classes).
- Compile with Adam optimizer (lr=1e-5), categorical cross-entropy loss, and categorical accuracy metric.
- Train for 40 epochs (batch size=32, 10 steps/epoch), using model checkpoints to save the best-performing model (./save/best_model.h5). Class weights address class imbalance.

1. Feature Extraction & Ranking

- Extract features from the VGG16 flatten layer for all resized images.
- **Aggregation:** For images with multiple crops, take the maximum similarity score to represent the original image.
- Compute cosine similarity between query and collection features, then rank collection images in descending order of similarity. Generate rankList.txt (./result/).

2.2 Histogram-based Color Feature Matching

2.2.1 Principle

Color histograms capture RGB channel pixel distribution—simple yet effective for low-level matching. Each channel's histogram (256 bins) is normalized, and Euclidean distance measures similarity between query and image histograms:

$$\text{Distance}(Q,F)=\sqrt{\sum_{i=1}^n (q_i - f_i)^2}$$

A smaller distance indicates greater color similarity.

2.2.2 Implementation Steps

1. **Preprocessing:** Reuse cropped/resized images from Section 2.1.1 to ensure consistency.
2. **Histogram Calculation:** Compute normalized RGB histograms (256 bins) for each image, concatenate into a single feature vector.
3. **Ranking:** Aggregate multi-crop histograms (max bin value), compute Euclidean distances, and rank images in ascending distance (descending similarity). Generate rankList_hist.txt (./result/).

3. Experimental Results

3.1 Top-10 Retrieval for Queries 1–5 (VGG16 Method)

Retrieved images are annotated with red bounding boxes (from JSON files; full-image boxes if no annotations). Key observations:

- **Query 1–2:** Top results contain instances of the same category as the query (e.g., cars, chairs) with consistent semantic features.
- **Query 3–5:** High relevance between queries and top results, confirming the method's ability to handle lighting/background variations.

Visualizations are saved as result.png (./result/).

3.2 Rank Lists

- rankList.txt: 50 rows (one per query) of 28,493 collection image ranks (VGG16 method).
- rankList_hist.txt: Same structure, using the histogram method.

4. Performance Analysis

4.1 Method Comparison

Method	Advantages	Disadvantages	Performance
VGG16-based	Captures semantic features; robust to variations	Computationally heavy; needs fine-tuning	High (accurate instance matching)
Histogram-based	Fast; low hardware requirements	Ignores shape/texture; poor for color variations	Moderate (color-only matching)

4.2 Key Influencing Factors

- **Preprocessing:** Accurate cropping and bicubic resizing preserve instance details; augmentation reduces overfitting.
- **Similarity Metric:** Cosine similarity outperforms Euclidean distance for high-dimensional VGG16 features.

5. Conclusion

Two instance search methods are successfully implemented. The VGG16-based method achieves superior performance by leveraging semantic features and accurate instance localization. The histogram method, while efficient, is limited to color-based matching.

Future improvements could integrate texture/shape features, use advanced models (ResNet/YOLO), or optimize similarity metrics. All required files (code, rank lists, visualizations) are organized for submission.